

LAB REPORT – EXPERIMENT 2

Dynamic Memory Allocation, Reallocation and Deallocation

Aim: To allocate a dynamic array using malloc(), reallocate it using realloc(), and deallocate it using free(). To demonstrate an unchecked allocation leading to NULL pointer dereference and a use-after-free through a dangling pointer, and then implement the corrected version.

1. Unsafe Program

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <unistd.h>
#include <sys/wait.h>

void null_dereference()
{
    size_t size = SIZE_MAX;
    int *ptr = malloc(size);

    printf("\nNULL pointer test\n");
    printf("Pointer: %p\n", (void *)ptr);
    printf("Allocated size: %zu bytes\n", size);

    /* malloc() return value is not checked */
    ptr[0] = 10;
}

void use_after_free()
{
    int n = 5;

    int *ptr = malloc(n * sizeof(int));

    printf("\nAfter malloc\n");
    printf("Pointer: %p\n", (void *)ptr);
    printf("Allocated size: %zu bytes\n", n * sizeof(int));

    for (int i = 0; i < n; i++)
        ptr[i] = i + 1;

    n = 10;

    ptr = realloc(ptr, n * sizeof(int));

    printf("\nAfter realloc\n");
    printf("Pointer: %p\n", (void *)ptr);
    printf("Allocated size: %zu bytes\n", n * sizeof(int));

    for (int i = 5; i < n; i++)
        ptr[i] = i + 1;

    int *dangling_ptr = ptr;

    free(ptr);

    printf("\nAfter free\n");
    printf("Pointer: %p\n", (void *)ptr);
    printf("Allocated size: 0 bytes\n");

    /* Use-after-free */
    printf("\nAccessing freed memory: %d\n", dangling_ptr[0]);
}

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        null_dereference();
        exit(0);
    }

    if (pid > 0)
    {
        wait(NULL);
        use_after_free();
    }
}
```

```
    return 0;  
}
```

Output of Unsafe Program

```
NULL pointer test  
Pointer: (nil)  
Allocated size: 18446744073709551615 bytes  
  
After malloc  
Pointer: 0x32dd32a0  
Allocated size: 20 bytes  
  
After realloc  
Pointer: 0x32dd36d0  
Allocated size: 40 bytes  
  
After free  
Pointer: 0x32dd36d0  
Allocated size: 0 bytes  
  
Accessing freed memory: 208339
```

The first output shows that `malloc()` returned `(nil)` for the extremely large allocation, while the later part shows memory being allocated, reallocated, freed, and then accessed through the dangling pointer. The value read after free is invalid because the memory has already been released.

2. Security Impact of Unsafe Code

NULL pointer dereference: Using a NULL pointer as if it referred to valid memory can crash the program and may cause denial of service.

Use-after-free: Accessing memory after it has been freed is a memory-safety vulnerability that can cause crashes, data corruption, or potentially more serious security problems depending on memory reuse.

3. Corrected Program

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n = 5;

    int *ptr = malloc(n * sizeof(int));

    if (ptr == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    printf("After malloc\n");
    printf("Pointer: %p\n", (void *)ptr);
    printf("Allocated size: %zu bytes\n", n * sizeof(int));

    for (int i = 0; i < n; i++)
        ptr[i] = i + 1;

    n = 10;

    int *temp = realloc(ptr, n * sizeof(int));

    if (temp == NULL)
    {
        printf("realloc failed\n");
        free(ptr);
        ptr = NULL;
        return 1;
    }

    ptr = temp;

    printf("\nAfter realloc\n");
    printf("Pointer: %p\n", (void *)ptr);
    printf("Allocated size: %zu bytes\n", n * sizeof(int));

    for (int i = 5; i < n; i++)
        ptr[i] = i + 1;

    printf("Array: ");

    for (int i = 0; i < n; i++)
        printf("%d ", ptr[i]);

    printf("\n");

    free(ptr);
    ptr = NULL;

    printf("\nAfter free\n");
    printf("Pointer: %p\n", (void *)ptr);
    printf("Allocated size: 0 bytes\n");

    return 0;
}
```

Output of Corrected Program

```

After malloc
Pointer: 0x242442a0
Allocated size: 20 bytes

After realloc
Pointer: 0x242446d0
Allocated size: 40 bytes
Array: 1 2 3 4 5 6 7 8 9 10

After free
Pointer: (nil)
Allocated size: 0 bytes

```

4. Corrections Made

Problem	Correction
Unchecked malloc()	Check whether the returned pointer is NULL.
Unchecked realloc()	Use a temporary pointer and check the result before updating ptr.
Use-after-free	Do not access memory after free().
Dangling pointer	Set ptr to NULL after free().
Uninitialized new memory	Initialize the newly allocated elements after successful realloc().

5. Result

The dynamic array was successfully allocated using malloc(), expanded using realloc(), and released using free(). The unsafe version demonstrated the required NULL-pointer and use-after-free cases. The corrected version checked allocation results, safely handled realloc(), initialized the array, freed the memory, and set the pointer to NULL. The corrected output shows the pointer as (nil) after deallocation.

6. Conclusion

The experiment demonstrates the importance of checking dynamic memory allocation results and avoiding access to freed memory. Proper pointer management prevents common C memory-safety errors.